

Labo 7 : Parallélisme des tâches

Bleuer Rémy

13.06.2026

Première partie : Analyse des k-mers

Compilation

```
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j
cd ..
```

Importer pi.txt (~50 Mo) pour les tests

```
python3 -c \
    "import random; print(''.join(random.choice('0123456789') for _ in range(50_000_000)), end='')" \
    > ./inputs/pi.txt
```

Exécution et vérification

```
./build/k-mer ./inputs/pi.txt 3 | sort > out_mono.txt
./build/k-mer-mt ./inputs/pi.txt 3 | sort > out_mt.txt
diff out_mono.txt out_mt.txt && echo "OK: resultats identiques"
```

Benchmark mono-threadé amélioré (plusieurs valeurs de k)

```
for k in 1 3 5 7 10; do
    echo "=== k=$k mono ==="
    for run in 1 2 3; do
        { time ./build/k-mer ./inputs/pi.txt $k > /dev/null ; } 2>> time_mono_k${k}.txt
    done
    cat time_mono_k${k}.txt
done
```

Benchmark multithreadé : scalabilité selon le nombre de threads

```
for k in 3 7; do
    for t in 1 2 4 8; do
        echo "=== k=$k threads=$t ==="
        export OMP_NUM_THREADS=$t
        for run in 1 2 3; do
            { time ./build/k-mer-mt ./inputs/pi.txt $k > /dev/null ; } 2>> time_mt_k${k}_t${t}.txt
        done
        cat time_mt_k${k}_t${t}.txt
    done
done
unset OMP_NUM_THREADS
```

Environnement de test

Élément	Valeur
Machine	Apple M3
Cœurs	8 (4 Performance + 4 Efficiency)
Mémoire	16 Go
Fichier d'entrée	50 millions de chiffres décimaux (~50 Mo)
Compilation	-O3 -mcpu=native, OpenMP via libomp
Méthode	3 exécutions par configuration, on retient le temps médian (real)

1. Introduction

Le programme analyse un fichier texte et compte la fréquence d'apparition de chaque sous-séquence de longueur k (k -mer). Pour une entrée de N caractères il existe $N - k + 1$ positions de départ, donc autant de k -mers à comptabiliser. La sortie est la liste de tous les k -mers distincts accompagnés de leur nombre d'occurrences.

Deux exécutables sont produits : une version mono-threadée améliorée (**k-mer**) et une version multithreadée OpenMP (**k-mer-mt**).

2. Inefficacités du code fourni et améliorations apportées

Le code de départ présentait deux défauts rédhibitoires.

Réouverture du fichier à chaque k -mer. La fonction `read_kmer` exécutait `fopen / fseek / fgetc x k / fclose` pour **chaque** position du fichier. Sur 50 millions de positions, cela représente 50 millions d'ouvertures de fichier : un coût d'I/O et d'appels système totalement disproportionné. La correction consiste à lire le fichier **une seule fois** entièrement en mémoire (`read_file`), puis à travailler directement sur le buffer : un k -mer à la position i est simplement le pointeur `data + i`, sans aucune copie ni relecture.

Recherche linéaire dans la table. La fonction `add_kmer` parcourait tout le tableau existant (`strcmp` sur chaque entrée) avant d'insérer ou d'incrémenter. La complexité globale était donc $O(N \times D)$ où D est le nombre de k -mers distincts. Pour de grands k , D devient énorme (jusqu'à ~10 millions d'entrées pour $k=7$), rendant l'algorithme quadratique et inutilisable. La correction remplace le tableau par une **table de hachage** (hash djb2, buckets en chaînage, redimensionnement quand le facteur de charge dépasse 0,75). L'insertion/incrémentation passe ainsi en $O(1)$ amorti, ramenant la complexité globale à $O(N)$.

Le comportement et l'interface en ligne de commande (`<input_file> <k>`) restent strictement identiques à l'original.

3. Performances de la version mono-threadée

Temps médian (**real**) sur 50 Mo :

k	Temps mono (s)	k-mers distincts (ordre de grandeur)
1	0,147	10
3	0,272	1 000
5	0,428	100 000
7	8,89	~10 000 000
10	30,3	~50 000 000 (approx. toutes positions distinctes)

L'analyse de cette baseline est révélatrice. Tant que le nombre de k-mers distincts reste petit ($k \leq 5$), le temps croît doucement : la table de hachage tient entièrement dans les caches CPU et chaque accès est quasi gratuit. Le travail est dominé par le simple parcours linéaire des 50 M de positions.

À partir de $k=7$, le temps fait un bond de 0,43 s à 8,9 s (x20). Le nombre de k-mers distincts (~10 M) dépasse alors largement la capacité des caches L1/L2 : chaque insertion provoque un défaut de cache vers la RAM. Le programme devient **memory-bound** plutôt que compute-bound. À $k=10$, presque chaque position produit un k-mer unique : la table contient des dizaines de millions d'entrées, les défauts de cache et la pression sur l'allocateur (`malloc` par entrée) dominent tout, d'où les 30 s.

Ce changement de régime, d'un coût dominé par le parcours à un coût dominé par les accès mémoire aléatoires dans la table, est le fait marquant de la baseline, et il conditionne entièrement le comportement de la version parallèle.

4. Stratégie de parallélisation

La parallélisation utilise OpenMP avec l'approche « **table locale par thread + fusion** » :

1. **Répartition du travail.** Les $N - k + 1$ positions de départ sont découpées en plages contiguës de taille égale, une par thread (`chunk = ceil(total_positions / nthreads)`). Chaque thread traite l'intervalle `[start, end)` de positions.
 2. **Comptage sans contention.** Chaque thread possède sa **propre table de hachage locale**. Pendant toute la phase de comptage, il n'y a donc **aucun verrou ni accès partagé**, les threads travaillent en totale indépendance, ce qui maximise le parallélisme.
 3. **Gestion des cas limites / chevauchement.** Le buffer complet est partagé en lecture seule et se termine par un sentinel `\0`. Un thread qui traite la position `i` lit `data[i .. i+k-1]`, ce qui peut empiéter de $k-1$ octets sur la plage du thread suivant, mais comme la lecture porte sur le buffer global, **aucun k-mer n'est perdu ni dupliqué** à la frontière. Chaque position de départ valide est traitée exactement une fois, par un seul thread. C'est un overlap de **lecture** (sûr), pas de comptage.
 4. **Fusion.** À la fin, chaque table locale est fusionnée dans une table globale au sein d'une section `#pragma omp critical`. Le coût de la fusion est proportionnel au nombre de k-mers **distincts** par thread, pas au nombre total de positions ; il est donc négligeable quand D est petit, mais devient significatif quand D est grand (voir analyse $k=7$).
-

5. Comparaison mono-threadée vs multithreadée

Cas $k=3$: comportement idéal

Threads	Temps (s)	Speedup vs 1 thread	CPU% observé
1	0,302	1,00x	~99 %
2	0,159	1,90x	~191 %
4	0,086	3,51x	~360 %
8	0,081	3,73x	~550 %

Avec seulement 1 000 k-mers distincts, chaque table locale tient intégralement dans le cache : le travail est purement compute-bound et parfaitement divisible. Le speedup est quasi linéaire jusqu'à 4 threads (3,5x pour 4 cœurs). Le passage de 4 à 8 threads n'apporte presque rien (3,5x $\rightarrow$ 3,7x) : les 4 threads supplémentaires s'exécutent sur les **cœurs Efficiency**, nettement plus lents que les cœurs Performance. C'est la signature classique d'une architecture hétérogène big.LITTLE : au-delà de 4 cœurs P, le gain marginal s'effondre.

Cas $k=7$: saturation mémoire (le cas instructif)

Threads	Temps real (s)	Temps user (s)	Speedup	CPU%
1	8,92	8,70	1,00x	~99 %
2	7,79	13,57	1,15x	~177 %
4	8,06	23,81	1,11x	~306 %
8	9,02	45,89	0,99x	~512 %

Ici, **ajouter des threads n'accélère pas le programme** : le temps real reste autour de 8–9 s quel que soit le nombre de threads. Pire, le temps **user expose** ($8,7 \text{ s} \rightarrow 46 \text{ s}$, soit $\times 5,3$ pour 8 threads) alors que le résultat calculé est identique. Le CPU% augmente bien (les cœurs tournent), mais ce travail supplémentaire est purement gaspillé.

L'explication tient à deux goulots d'étranglement :

- **Bande passante mémoire saturée.** Avec $\sim 10 \text{ M}$ de k-mers distincts, chaque table locale fait plusieurs centaines de Mo et déborde très largement des caches. Chaque insertion est un accès aléatoire à la RAM. Plusieurs threads se partagent la **même bande passante mémoire** : ils ne se parallélisent pas, ils se mettent en file d'attente sur le contrôleur mémoire. Le temps user gonfle car chaque cœur passe son temps à attendre des lignes de cache (temps compté comme « actif »).
- **Coût de la fusion en section critique.** À 8 threads, ce sont 8 tables de $\sim 10 \text{ M}$ d'entrées à fusionner séquentiellement dans la table globale. Cette phase, sérialisée par `omp critical`, ne profite pas du parallélisme et son poids croît avec le nombre de threads.

Conclusion de la comparaison. La parallélisation par découpage est très efficace **quand le problème est compute-bound et que l'empreinte mémoire par thread tient en cache** (petit k) : on atteint alors un speedup proche du nombre de cœurs Performance. Dès que le problème devient memory-bound (grand k , grand nombre de k-mers distincts), ajouter des threads est contre-productif. La bande passante mémoire devient le facteur limitant, et la duplication des tables locales aggrave encore la pression mémoire. Sur cette architecture M3, le sweet spot pratique est de **4 threads** (cœurs Performance), au-delà duquel les cœurs Efficiency et la saturation mémoire annulent tout gain.

Des pistes d'amélioration possibles seraient : une table de hachage partagée par verrous fins (sharding) pour éviter la fusion finale coûteuse, ou une fusion arborescente parallèle (réduction) plutôt que séquentielle, afin d'atténuer le goulot du `critical` dans le cas grand k .

Deuxième partie : Activité Pan-Tompkins

Compilation et exécution

```
mkdir build
cd build
cmake ..
make
cd ..
./build/ecg_streaming 80bpm0.csv resultat_streaming.json
```

Benchmark

```
hyperfine --warmup 20 -N \
'./build/ecg_streaming 80bpm0.csv /dev/null' \
--export-json bench_streaming.json
```

1. Introduction

Le code du lab01 implémente la chaîne de traitement Pan-Tompkins pour détecter les pics R dans un signal ECG. Le pipeline se déroule en cinq étapes séquentielles sur le signal complet : filtre passe-haut (suppression de la dérive lente), dérivée du premier ordre (accentuation des transitions rapides du QRS), mise au carré (rectification et

accentuation non-linéaire), intégration sur fenêtre glissante (lissage de l'énergie), puis détection des pics avec un seuil adaptatif et une période réfractaire. Sur le fichier de test (500 Hz, 7500 échantillons, 15 secondes), il détecte 20 pics R à 79,9 BPM avec des intervalles RR de 0,75 s.

L'adaptation demandée consiste à passer d'un traitement monolithique (tout le signal d'un coup) à un **traitement par paquets de 1000 échantillons** avec un **chevauchement de 250 échantillons** entre paquets consécutifs, simulant un flux continu de données.

2. Partie du code parallélisée

La partie parallélisée est le **traitement de chaque paquet** : les étapes 1 à 4 du pipeline (passe-haut, dérivée, carré, MWI) sont indépendantes entre paquets dès lors que l'état des filtres est préservé. La détection des pics (étape 5) ne peut pas être parallélisée directement car le seuil adaptatif et la période réfractaire créent une dépendance séquentielle entre pics consécutifs.

3. Stratégie de parallélisation

Décomposition en paquets avec overlap. Le signal est découpé en paquets qui avancent d'un pas $STEP = 750$ échantillons à chaque itération, chaque paquet couvrant $CHUNK_SIZE = 1000$ échantillons. Les 250 premiers échantillons de chaque paquet (sauf le premier) correspondent à la fin du paquet précédent : c'est la zone de chevauchement.

Préservation de l'état des filtres entre paquets. Les filtres passe-haut et MWI sont des moyennes glissantes à mémoire : leur somme courante (`hp_sum`, `mwi_sum`) et leur largeur de fenêtre effective (`hp_w`, `mwi_w`) sont stockées dans une structure `StreamState` persistante d'un paquet à l'autre. Sans cela, chaque paquet repartirait avec une somme nulle, produisant une discontinuité visible aux frontières.

Dédoublonnage des pics dans la zone de chevauchement. Puisque la zone $[0, OVERLAP-1]$ de chaque paquet a déjà été traitée par le paquet précédent, un pic détecté dans cette zone serait compté deux fois. La règle est simple : un pic local à l'indice i dans un paquet n'est accepté que si $i \geq OVERLAP$ (pour tous les paquets sauf le premier). Le seuil adaptatif et la période réfractaire sont quand même mis à jour pour les pics de la zone chevauchante (ignorés comme résultats mais utilisés pour faire évoluer correctement les paramètres adaptatifs).

Période réfractaire inter-paquets. La variable `last_r_global` stocke l'indice global du dernier pic R accepté. Le test de période réfractaire est fait en coordonnées globales (`global_i - last_r_global < refract`) pour éviter qu'un pic en début de paquet ne soit accepté alors qu'il est trop proche d'un pic détecté à la fin du paquet précédent.

Calibration initiale du seuil. Le seuil adaptatif de Pan-Tompkins est initialisé à 25 % du `max_mwi`. En mode streaming, ce `max_mwi` ne peut pas être connu à l'avance. La solution retenue est d'exécuter le pipeline complet une fois sur le premier paquet (sans état persistant, uniquement pour mesurer le `max_mwi` des 1000 premiers échantillons), puis d'initialiser le seuil à partir de cette valeur. C'est un surcoût d'une passe sur 1000 échantillons, négligeable devant le traitement total.

4. Performances et évaluation

Version	Temps médian	Temps moyen	Écart-type	Mémoire RSS
Original (bloc complet)	4,74 ms	4,76 ms	0,16 ms	3 600 Ko
Streaming par paquets	4,89 ms	5,01 ms	1,72 ms	3 470 Ko

Les temps sont mesurés avec `hyperfine` sur Apple M3, signal de 7500 échantillons à 500 Hz.

Temps d'exécution. Les deux versions sont quasi-équivalentes en termes de temps médian (~4,7 ms vs ~4,9 ms), ce qui est attendu, le travail total de calcul est identique. Le mode streaming introduit un léger surcoût lié à la boucle de gestion des paquets et à la passe de calibration initiale, mais ce surcoût est complètement noyé dans le temps de chargement du CSV et d'écriture du JSON. L'écart-type nettement plus élevé de la version streaming (1,72 ms vs 0,16 ms) et le pic à 38,8 ms s'expliquent par les interruptions du scheduler OS sur des intervalles de traitement très courts : phénomène classique sur des benchmarks de l'ordre de quelques millisecondes.

Mémoire. La version streaming utilise légèrement moins de mémoire (~130 Ko de moins). L'original alloue quatre buffers de `MAX_SAMPLES` doubles dans `ecg_create()`, quelle que soit la taille réelle du signal. Le streaming travaille sur des buffers de taille fixe `CHUNK_SIZE = 1000`, indépendamment de la longueur totale du signal : avantage qui deviendrait significatif pour des flux très longs.

Résultats fonctionnels. Les deux versions produisent exactement les mêmes 20 pics R aux mêmes indices (330, 705, 1080, ... 7461) et les mêmes intervalles RR de 0,75 s. La stratégie de chevauchement et de dédoublement est donc correcte.

5. Intérêt et limites de la parallélisation

Sur un signal de 7500 échantillons, **la parallélisation est peu pertinente** : le signal est trop court pour que le découpage en paquets crée un parallélisme exploitable, et le coût de création des threads OpenMP surpasserait le gain de calcul. Le vrai bénéfice de l'approche par paquets est ailleurs :

Bénéfice réel : passage à l'échelle mémoire. En mode flux continu (enregistrement Holter de 24h, monitoring temps réel), il est impossible de charger tout le signal en mémoire. Le mode streaming traite les données avec une empreinte mémoire $O(1)$ par rapport à la durée du signal, ce qui est le prérequis fonctionnel à tout traitement long.

Parallélisme potentiellement utile : paquets indépendants. Si plusieurs dérivations ECG (12 leads dans le fichier de test) doivent être analysées simultanément, chaque lead peut être traité par un thread différent sur son propre `StreamState`. Ce parallélisme est naturel, sans contention, et donnerait un speedup proche de 12x (nombre de leads) sur une machine multi-cœurs. C'est le cas d'usage où OpenMP aurait un intérêt réel ici.

Limite principale. La détection des pics (étape 5) reste séquentielle à l'intérieur d'un lead à cause des dépendances du seuil adaptatif. Un pipeline producteur-consommateur (un thread produit les buffers MWI, un autre détecte les pics) pourrait théoriquement recouvrir calcul et détection, mais la latence de communication entre threads annulerait probablement le gain sur des paquets de 1000 échantillons.